Global Variables and Top-Level Instance Variables

Global variables can be accessed by any scope:

```
def a_scope
  $var = "some value"
end

def another_scope
  $var
end

a_scope
another_scope # => "some value"
```

The problem with global variables is that every part of the system can change them, so in no time you'll find it difficult to track who is changing what. For this reason, the general rule is this: when it comes to global variables, use them sparingly, if ever.

You can sometimes use a top-level instance variable in place of a global variable. These are the instance variables of the top-level main object, described in *The Top Level*, on page 35:

```
@var = "The top-level @var"

def my_method
  @var
end

my_method # => "The top-level @var"
```

You can access a top-level instance variable whenever main takes the role of self, as in the previous example. When any other object is self, the top-level instance variable is out of scope.

```
class MyClass
  def my_method
    @var = "This is not the top-level @var!"
  end
end
```

Being less universally accessible, top-level instance variables are generally considered safer than global variables—but not by a wide margin.

What happens when the program creates a MyClass object and calls my_method twice? The first time the program enters my_method, it opens a new scope and defines a local variable, v3. Then the program exits the method, falling back to the top-level scope. At this point, the method's scope is lost. When the program calls my_method a second time, it opens yet another new scope, and it defines a new v3 variable (unrelated to the previous v3, which is now lost). Finally, the program returns to the top-level scope, where you can see v1 and obj again. Phew!